Assignment 3 Report – Transformer for IMDb Sentiment Classification

Wimukthi Rajapaksha (a1988198)
COMP6026 Applied Artificial Intelligence and Machine Learning 2

October 2026

1 Introduction

A two-layer encoder-only Transformer, implemented from scratch in PyTorch, reaches a test accuracy of 0.8669 and an F1-score of 0.8691 on the 25,000-review IMDb test set with the configuration the assignment specifies. The best variant, selected by validation loss, reaches 0.8805 accuracy and 0.8844 F1. No prebuilt attention or Transformer modules were used.

The task is binary sentiment classification of IMDb movie reviews (positive = 1, negative = 0) [2]. The dataset contains 50,000 reviews, split evenly into a 25,000-review training set and a 25,000-review test set, and both splits are balanced between the two classes. This report describes the data pipeline, the model, the training and model-selection procedure, the results, and seven experiments that address training efficiency and performance; two of them were repeated, together with the baseline, over three seeds. All results come from the submitted run of the notebook on a Google Colab NVIDIA T4 GPU, except comparisons with earlier development runs, which are marked as such. The report closes with answers to the two assigned questions.

2 Data processing

The official training set was split 80/20 into training and validation subsets. The official test set was held out and used only once, for the final evaluation of the selected model.

Split	Source	Reviews	Positive share	Used for
Train	Official train, shuffled (seed 42)	20,000	0.500	Gradient updates, vocabulary
Validation	Official train, held-out 20%	5,000	0.501	Model selection
Test	Official test, untouched	25,000	0.500	Final evaluation only

Table 1: Data splits.

The preprocessing pipeline has four steps:

1. **Loading.** The dataset (`stanfordnlp/imdb`) was loaded with the Hugging Face `datasets` library, as required.
2. **Tokenisation.** HTML tags were removed, since many reviews mark line breaks with `<br />` tags. The text was then lowercased and split into word and punctuation tokens using torchtext's `basic_english` rules. torchtext is deprecated, and its final release (0.18) supports only PyTorch 2.3. The notebook therefore uses torchtext's tokenizer when it can be imported and otherwise an exact re-implementation whose twelve regular-expression rules are copied verbatim from torchtext's source (`torchtext/data/utils.py`, function

`_basic_english_normalize`), so tokenisation is identical either way. The notebook's first code cell attempts to install torchtext 0.18 with PyTorch 2.3; in the submitted run this failed because Colab's runtime uses Python 3.13, for which no torchtext packages exist, so the re-implementation was used.

3. **Vocabulary.** The vocabulary was built from the training subset only, so no information leaks from the validation or test data. Tokens occurring at least twice were kept, capped at the 20,000 most frequent. Index 0 is reserved for `<pad>` and index 1 for `<unk>`.

4. **Encoding and padding.** Each review was mapped to token ids, with unknown tokens mapped to `<unk>`, truncated to 256 tokens, and right-padded with `<pad>` to a uniform length of 256. The true length of each review was stored for masking and batching.

Length analysis. Training reviews have a median of 203 tokens and a mean of 271 (90th percentile 530, maximum 2,752). 36.2% of reviews exceed 256 tokens, and truncation at 256 discards 28.4% of all training tokens; at 512 the figures are 10.7% and 8.9%. On the validation set, 2.85% of tokens map to `<unk>`. In fixed-length batches, 24.1% of all positions are padding. These figures motivated the masking and efficiency experiments in Section 6 (Figure 1).

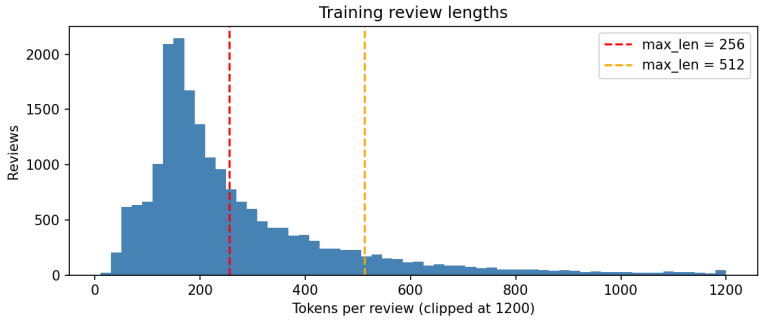


Figure 1: Distribution of review lengths in the training subset.

Data integrity checks. No review is empty, and no training review occurs with both labels. Table 2 lists the exact duplicates (after lowercasing and whitespace normalisation).

Duplicate copies within a split		Distinct reviews shared between splits	
Train	66	Train and validation	28
Validation	2	Train and test	93
Test	199	Validation and test	30

Table 2: Duplicate reviews found by the data integrity checks.

These overlaps are too small to affect any conclusion. The 93 test reviews that also occur in the training set are 0.37% of the test set. Even if the model classified every one of them correctly only because it had seen it during training, they would raise test accuracy by at most $93 \times (1 - 0.867)/25{,}000 \approx 0.05$ percentage points. The duplicates were therefore kept, so that the official test set stays unchanged and the results remain comparable with other work on IMDb.

3 Model architecture

The model follows the required pipeline: input, embedding, positional encoding, multi-head attention, feed-forward network, classification head, output. It stacks two encoder blocks (Figure 2) and has 2,825,218 trainable parameters, most of them (2,560,000) in the embedding table. Every component was written from basic tensor operations and `nn.Linear`, `nn.Embedding` and `nn.LayerNorm`.

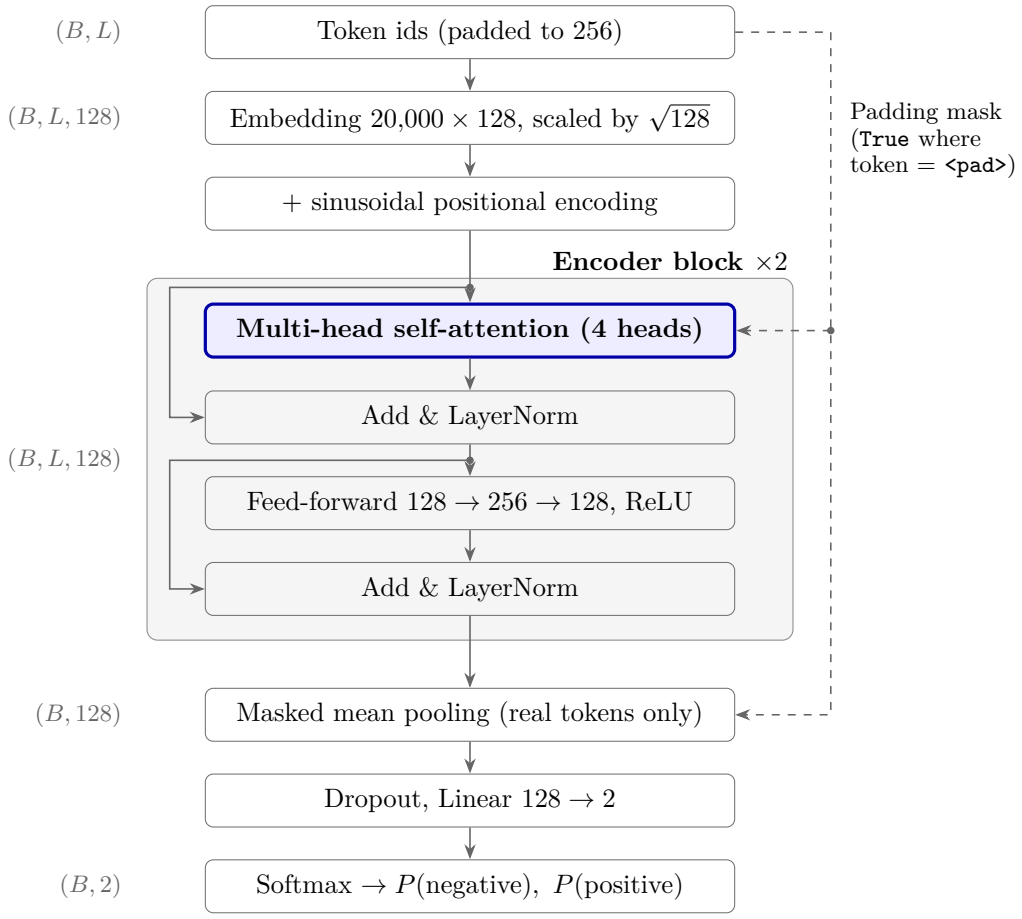


Figure 2: Model architecture: two Post-LN encoder blocks. The padding mask derived from the token ids is applied in every attention layer and in the pooling step; the left-hand arrows are the residual connections.

Component	Implementation	Shape
Input embedding	`nn.Embedding(20000, 128)`, initialised with std $128^{-0.5}$ and scaled by $\sqrt{128}$	$(B, L) \to (B, L, 128)$
Positional encoding	Sinusoidal, stored as a non-trainable buffer and added to the embeddings	$(B, L, 128)$
Multi-head self-attention	W_q, W_k, W_v, W_o projections; 4 heads of $d_k = 32$; scaled dot-product attention with a padding mask	$(B, L, 128)$
Feed-forward network	Linear(128, 256) → ReLU → Dropout → Linear(256, 128)	$(B, L, 128)$
Encoder block (×2)	Post-LN: LayerNorm(x + Dropout(Sublayer(x))) around attention and around the FFN	$(B, L, 128)$
Pooling	Mean over non-padding positions only	$(B, 128)$
Classification head	Dropout → Linear(128, 2) → softmax	$(B, 2)$

Table 3: Model components (B = batch size, L = sequence length).

3.1 Mathematical formulation

For a review with token ids $x_1, \ldots, x_L$ and padding indicator $m_i = 1$ for real tokens and 0 for `<pad>`, with $d = 128$, $H = 4$ heads and $d_k = d/H = 32$ (dropout omitted for clarity):

$$\mathbf{h}_i^{(0)} = \sqrt{d}\, E[x_i] + \mathrm{PE}_i \tag{1}$$

$$\mathrm{PE}_{i,2j} = \sin\!\left(i/10000^{2j/d}\right), \qquad \mathrm{PE}_{i,2j+1} = \cos\!\left(i/10000^{2j/d}\right) \tag{2}$$

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\!\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \qquad M_{ij} = \begin{cases} 0 & m_j = 1 \\ -\infty & m_j = 0 \end{cases} \tag{3}$$

$$\mathrm{head}_h = \mathrm{Attention}(XW_q^h, XW_k^h, XW_v^h) \tag{4}$$

$$\mathrm{MHA}(X) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_H)\, W_o \tag{5}$$

$$\mathrm{FFN}(\mathbf{x}) = W_2\, \mathrm{ReLU}(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 \tag{6}$$

$$X' = \mathrm{LN}(X + \mathrm{MHA}(X)), \qquad X^{\mathrm{out}} = \mathrm{LN}(X' + \mathrm{FFN}(X')) \tag{7}$$

$$\mathbf{z} = \frac{\sum_i m_i\, \mathbf{h}_i^{(2)}}{\sum_i m_i}, \qquad \mathbf{p} = \mathrm{softmax}(W_c\mathbf{z} + \mathbf{b}_c) \tag{8}$$

$$\mathcal{L} = -\big[\, y \log p_1 + (1-y)\log(1-p_1)\,\big] \tag{9}$$

In the code, $-\infty$ is the most negative representable value of the score's data type, so a fully padded row can never produce `NaN`.

Backward propagation uses PyTorch autograd. For a mini-batch of $B = 32$ reviews with logits $\mathbf{s}^{(n)} = W_c\mathbf{z}^{(n)} + \mathbf{b}_c$ and one-hot labels $\mathbf{y}^{(n)}$, the loss is averaged over the batch, and the gradient of softmax followed by cross-entropy takes a simple form:

$$\mathcal{L}_{\mathrm{batch}} = \frac{1}{B}\sum_{n=1}^{B} \mathcal{L}^{(n)}, \qquad \frac{\partial \mathcal{L}_{\mathrm{batch}}}{\partial \mathbf{s}^{(n)}} = \frac{1}{B}\left(\mathbf{p}^{(n)} - \mathbf{y}^{(n)}\right) \tag{10}$$

Because of the $1/B$ factor, the gradient is the average over the batch, so its scale does not depend on the batch size. Every parameter θ is then updated by Adam [3] with $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ and $g_t = \nabla_\theta \mathcal{L}_{\mathrm{batch}}$ (after clipping the global norm of all gradients to 1.0):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2 \tag{11}$$

$$\theta_t = \theta_{t-1} - \eta\, \frac{m_t/(1-\beta_1^t)}{\sqrt{v_t/(1-\beta_2^t)} + \epsilon} \tag{12}$$

Three design decisions shape the implementation:

- **Padding mask.** Scores for `<pad>` keys are set to the most negative representable value before the softmax, so padding receives zero attention weight. The mask has shape $(B, 1, 1, L)$ and broadcasts over heads and query positions. A unit test in the notebook confirms that adding padding to a review leaves its prediction unchanged.
- **Masked mean pooling.** The sequence representation is the average of the encoder outputs over real tokens. This was preferred to a `[CLS]` token because it needs no extra token, and every position contributes directly to the gradient.
- **Output layer and loss.** The head produces two logits; a softmax over them gives the class probabilities, as the brief requires. Cross-entropy over two softmax classes equals the binary cross-entropy in Equation (9) with $p_1 = P(\text{positive})$, so the model meets both the softmax-output and binary-cross-entropy requirements. `nn.CrossEntropyLoss` computes this from log-softmax, which is numerically stable.

The embeddings are initialised small and then multiplied by $\sqrt{d_{\text{model}}}$, so they start at unit scale, comparable to the positional encoding. With PyTorch's default $\mathcal{N}(0, 1)$ initialisation, the same scaling would make the embeddings about 11 times larger than the positional signal and drown it out.

The number of classes, the pooling method (mean or `[CLS]`), Post-LN or Pre-LN blocks, the feed-forward activation and the positional-encoding base are all configuration options; a unit check in the notebook builds and runs the alternative settings.

4 Training and model selection

The model was trained with the required configuration, and the checkpoint with the lowest validation loss was selected for testing.

Setting	Value
Optimiser	Adam [3], learning rate 0.001
Batch size	32 (mini-batch processing; 625 updates per epoch)
Epochs	10
Loss	Cross-entropy over two softmax outputs (binary cross-entropy)
Regularisation	Dropout 0.1 on embeddings, attention weights, sublayer outputs and the head
Gradient clipping	Global L2-norm capped at 1.0
Seed	42 for Python, NumPy and PyTorch
Hardware	Google Colab, NVIDIA Tesla T4 GPU (16 GB); PyTorch 2.11.0 (CUDA)

Table 4: Training configuration of the baseline.

Gradients were computed by backpropagation of the loss through all layers (PyTorch autograd), and Adam updated every weight. Gradient clipping was added because a Post-LN Transformer trained at a fixed learning rate of 0.001, with no warm-up, can produce occasional large gradient spikes. Clipping bounds the size of each update without changing the required learning rate. The seed fixes the initial weights, the dropout masks and the batch order, and cuDNN runs in deterministic mode; some GPU kernels still sum floating-point values in a varying order, so repeating a run could in principle change the last digits of the metrics. In practice, an earlier complete run of the same notebook on the T4, in a separate Colab session and not part of the submitted notebook, gave exactly the same accuracy, loss and memory values as the submitted run; only the timings differed, by up to about 2% for epoch times and up to about 4% for the micro-benchmark's step times (Section 6.2).

Model selection. After each epoch the model was evaluated on the 5,000-review validation subset, and its weights were saved whenever the validation loss reached a new minimum. The saved checkpoint, rather than the final-epoch model, was evaluated on the test set. Validation loss was chosen over validation accuracy because it also penalises confident wrong predictions, so it signals the onset of overfitting earlier. The same criterion chose the final model across all experiments in Section 6: the run with the lowest best-checkpoint validation loss. The test set played no part in any training or selection decision.

5 Results

The selected baseline checkpoint (epoch 1) achieves 0.8669 accuracy and 0.8691 F1 on the test set, within the 80–90% target range.

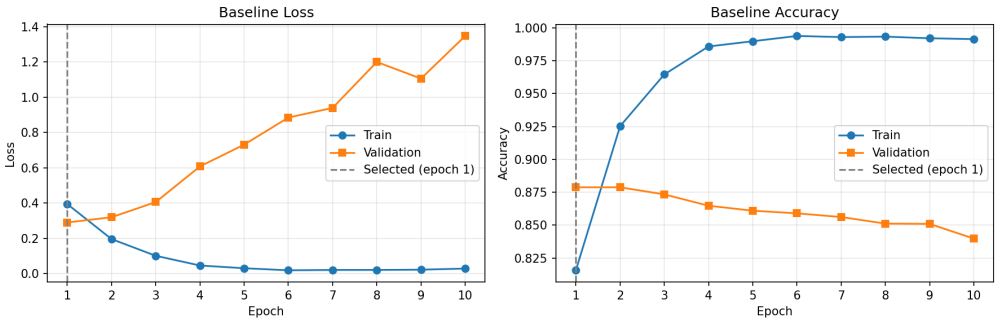


Figure 3: Baseline loss and accuracy per epoch, training versus validation.

Epoch	Train loss	Train accuracy	Val loss	Val accuracy
1	0.3946	0.8157	**0.2902**	**0.8788**
2	0.1962	0.9253	0.3199	0.8788
3	0.1014	0.9646	0.4059	0.8734
4	0.0465	0.9859	0.6080	0.8648
5	0.0309	0.9899	0.7314	0.8610
6	0.0192	0.9939	0.8849	0.8590
7	0.0213	0.9930	0.9394	0.8562
8	0.0213	0.9933	1.1999	0.8512
9	0.0228	0.9921	1.1056	0.8510
10	0.0290	0.9915	1.3472	0.8398

Table 5: Baseline training history (about 12.5 s per epoch). Bold: the selected checkpoint.

5.1 Training and validation curves

Training loss falls from 0.3946 to 0.0192 by epoch 6 and then stays between 0.02 and 0.03 (Figure 3). Validation loss reaches its minimum of 0.2902 at epoch 1 and then rises to 1.3472, while training accuracy stays above 0.99 from epoch 6. This divergence shows the model memorising the training reviews. Selecting the epoch-1 checkpoint avoids reporting the overfitted final model, whose validation accuracy is 0.8398 compared with 0.8788 at the selected epoch. Validation accuracy is identical at epochs 1 and 2 (0.8788), but validation loss has already risen at epoch 2 (0.3199): the model is becoming more confident on the reviews it gets wrong, which accuracy does not show. This is why validation loss is the selection criterion.

5.2 Test-set metrics

Metric (test, 25,000)	Value	Class	Precision	Recall	F1	Support
Accuracy	0.8669	Negative (0)	0.8799	0.8498	0.8646	12,500
Precision	0.8547	Positive (1)	0.8547	0.8840	0.8691	12,500
Recall	0.8840	Macro average	0.8673	0.8669	0.8668	25,000
F1-score	0.8691					

Table 6: Baseline test metrics (left) and class-wise metrics (right).

Table 6 lists the test metrics. The confusion matrix (Figure 8a) contains 10,622 true negatives, 1,878 false positives, 1,450 false negatives and 11,050 true positives. Precision is lower than recall, so the model errs towards predicting positive.

The lean is mild. Negative recall is 0.85, so 15.0% of negative reviews are classified as positive, whereas 11.6% of positive reviews are missed (recall 0.88). The lean is not a fixed property of the data: the regularised run with a learning-rate schedule leans strongly the other way (precision

0.92, recall 0.73; Section 6.4), so the direction depends on where a particular training run places its decision boundary. A plausible contributing factor is that many negative reviews praise individual aspects of a film, such as the actors or the photography, before rejecting it, so they contain strongly positive vocabulary. Section 5.4 shows that moving the threshold removes most of the imbalance.

Validation scores are optimistic. In all 14 training runs in the notebook, the selected checkpoint's validation accuracy is higher than its test accuracy, by 0.8 to 3.4 percentage points (baseline: 0.8788 against 0.8669). Two effects contribute. The checkpoint is chosen because it scores best on these 5,000 validation reviews, which favours checkpoints that happen to suit them. In addition, the validation reviews come from the official training set and therefore cover the same films as the training reviews, whereas the official test set contains a disjoint set of films [2], so film-specific words such as actor and character names most likely help on validation but not on test. Validation scores are therefore used here to compare models, not as estimates of test accuracy.

5.3 Error analysis

The model is less accurate on reviews it could not read in full: accuracy is 0.8884 on test reviews of at most 256 tokens versus 0.8265 on the 8,699 truncated reviews. Figure 4 breaks this down further: up to 256 tokens the three models are within about 1.5 percentage points of each other (0.877 to 0.890), but the baseline falls to 0.81 at 385–512 tokens and to 0.77 above 768 tokens.

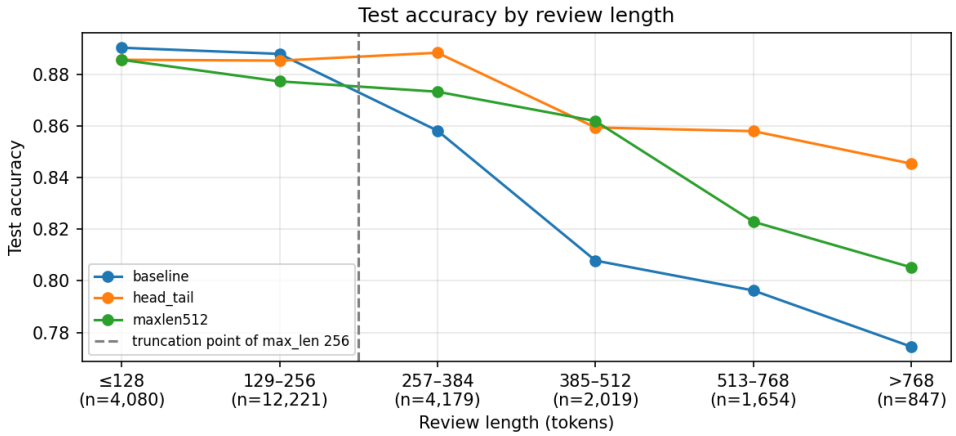


Figure 4: Test accuracy by review length for the baseline and the two longer-context variants. The dashed line marks the baseline's 256-token truncation point.

Confident mistakes. The three mistakes the model makes with the highest confidence (notebook §4.5) are labelled negative but read as positive. One calls the film "one of the best Kung fu movies in the history of Cinema", another "a true classic", and in the third the reviewer says they have "developed an enjoyment" of the film after watching it nearly five times. IMDb labels were derived from the reviewers' star ratings (4 or below negative, 7 or above positive) [2], so these are most likely rating–text mismatches rather than model errors: the model's prediction agrees with the text. Part of the measured error is therefore a floor set by label noise, which no model can remove.

Very long reviews. Above 768 tokens, head + tail truncation (0.85) is more accurate than max_len 512 (0.81), although it reads half as many tokens, and it is also ahead at 513–768 tokens (0.86 against 0.82). Even 512 tokens cuts off the end of the longest reviews, whereas head + tail always keeps the final 192 tokens. This supports the assumption that reviewers often state their verdict at the end.

Figure 5 does not show attention concentrating on the clearest sentiment word. The most-attended tokens are two occurrences of "the", at a little over twice the uniform weight of 1/43,

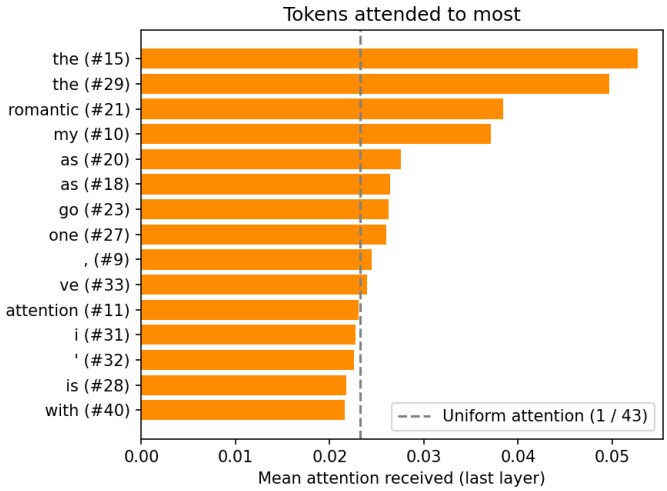


Figure 5: Last-layer attention for a short negative review, with the uniform-attention reference line.

followed by "romantic" and "my" at about 1.6 times the uniform weight. "Worst" is not among the 15 most-attended tokens, so it receives less than the uniform weight. The model still classifies the review confidently as negative ($P(\text{positive}) = 0.014$). With mean pooling, the output of every token contributes equally to the classification, so sentiment can be carried by the token representations themselves (built by the embeddings, the value projections and the feed-forward layers) rather than by sharp attention patterns. Head-averaged attention weights are therefore a weak explanation of this model's predictions.

5.4 Decision threshold

Model	Threshold	Accuracy	Precision	Recall	F1	FP	FN
Baseline	0.50 (default)	0.8669	0.8547	0.8840	0.8691	1,878	1,450
	0.55 (tuned on validation)	0.8660	0.8696	0.8613	0.8654	1,615	1,734
Head + tail	0.50 (default)	0.8805	0.8565	0.9142	0.8844	1,915	1,072
	0.60 (tuned on validation)	0.8818	0.8833	0.8800	0.8816	1,454	1,500

Table 7: Effect of the decision threshold on $P(\text{positive})$. The threshold was chosen to maximise validation accuracy and then applied once to the test set.

Moving the threshold barely changes accuracy (-0.09 percentage points for the baseline and $+0.13$ for head + tail), but it brings the two error types close together: 1,615 false positives against 1,734 false negatives for the baseline, which slightly overshoots, and 1,454 against 1,500 for head + tail. A threshold changes only the cut-off, not how the reviews are ranked by $P(\text{positive})$. The positive lean is therefore a calibration offset, a shifted decision boundary, rather than a lack of discriminative power. Because the threshold was tuned on the validation set and applied once to the test set, this result is not inflated by the test data; the baseline's small loss shows that a threshold tuned on 5,000 validation reviews does not transfer exactly to the test set.

6 Challenges and solutions

Several challenges affected training efficiency and performance, and each solution was tested in a controlled experiment. Every run used the same seed, data split, vocabulary, model size and optimiser settings as the baseline, and changed only the factors named in Table 8. All runs were evaluated with their own best-validation checkpoint (source: `experiment_summary.csv`; Figure 6).

On the T4, epoch times varied by about 1% within one session (the three baseline seeds took 12.52–12.66 s per epoch) and by up to about 2% between two separate sessions, while step times in the micro-benchmark varied by up to about 4% between sessions (Section 4). Only timing differences larger than about 5% are therefore treated as real; the hardware-independent attention-work column (sum of $B \cdot L^2$ per epoch, relative to the baseline) is reported alongside. Mixed precision (AMP) runs the forward pass in fp16 with gradient scaling and is available only on CUDA GPUs.

Experiment	Change from baseline	Epoch (s)	Speed-up	Attn. work	Train mem. (MB)	Best epoch	Best val loss	Test acc.	Test F1
Baseline	– (max_len 256, head truncation, fixed padding, mask, fp32)	12.5	1.00×	1.00	289	1	0.2902	0.8669	0.8691
No padding mask	Mask removed from attention and pooling	11.4	1.10×	1.00	271	1	0.2945	0.8598	0.8657
Dynamic padding	Per-batch padding (to a multiple of 16) + length bucketing	9.8	1.27×	0.67	271	1	0.2999	0.8570	0.8646
Dynamic padding + AMP	+ fp16 mixed precision	8.1	1.55×	0.67	208	1	0.3013	0.8580	0.8658
Head + tail	First 64 + last 192 tokens of long reviews	12.6	0.99×	1.00	271	1	**0.2665**	**0.8805**	**0.8844**
max_len 512	512 tokens with dynamic padding and AMP	11.6	1.08×	1.30	617	1	0.2759	0.8706	0.8768
Regularised	Dropout 0.3, AdamW weight decay 0.1	12.7	0.99×	1.00	272	1	0.2904	0.8639	0.8641
Regularised + schedule	+ linear warm-up (10%), cosine decay	12.7	0.99×	1.00	272	2	0.3379	0.8326	0.8130

Table 8: Controlled experiments (seed 42, Colab T4). Training memory is the GPU memory allocated during training above the level before training.

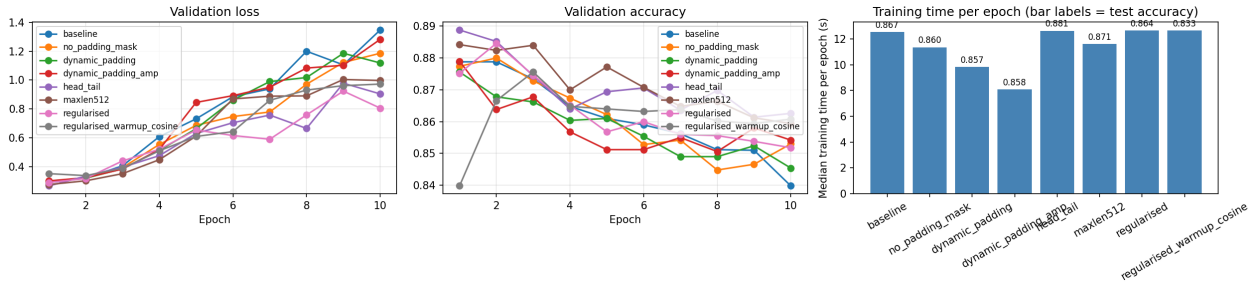


Figure 6: Comparison of the experiments.

6.1 Padding dominating the input (performance)

Challenge. 24.1% of the positions in a fixed-length batch are `<pad>`. Without a mask, every token attends to these positions and the pooled vector averages over them, so short reviews are represented mostly by padding.

Solution and evidence. Masking padded keys before the softmax and pooling over real tokens only changed test accuracy from 0.8598 (no mask) to 0.8669 (baseline), and the best validation loss from 0.2945 to 0.2902. By epoch 10 the unmasked model's validation loss was in fact lower (1.1859 against 1.3472).

The hypothesis was not confirmed for accuracy: the 0.7-point difference is inside the seed-to-seed variation of the baseline (standard deviation 0.0117, Table 10). The unmasked model can still work because every padding position carries the same `<pad>` embedding plus its positional encoding, so the model can learn to give these positions little influence; with 24.1% padding on average, real tokens still dominate the mean. The mask is nevertheless the correct implementation. Without it, a review's prediction depends on how much padding its batch adds, which the padding-invariance unit test rules out and which would make predictions differ between fixed and dynamic padding. The mask has a measurable cost: the unmasked run was 10% faster per epoch (11.4 s against 12.5 s), above the 5% threshold for timing noise, most likely because the mask is applied to the full $B \times H \times L \times L$ score tensor in every attention layer.

6.2 Computation wasted on padding (efficiency)

Challenge. With fixed padding every batch has shape 32×256, however short its reviews are. Attention cost grows with the square of the sequence length, so padding consumes a large share of the computation.

Solution and evidence. Dynamic padding trims each batch to its longest review, and length bucketing groups reviews of similar length into the same batch while keeping batch order shuffled. Together they cut the attention work per epoch by a third (0.67 of the baseline) and the training time per epoch from 12.5 s to 9.8 s (1.27× faster). Adding fp16 mixed precision with gradient scaling, which uses the T4's tensor cores, cut it further to 8.1 s: 1.55× faster than the baseline and 1.22× faster than fp32 dynamic padding. AMP also reduced training memory from 271 MB to 208 MB (−23%). Test accuracy was 0.8570 with dynamic padding and 0.8580 with AMP, against 0.8669 for the baseline.

The one-point accuracy difference is within the baseline's seed standard deviation (0.0117), and the best validation accuracy is close to the baseline's (0.8756 against 0.8788), so it is not clear evidence that dynamic padding harms accuracy. Both runs lean more strongly towards positive predictions than the baseline (recall 0.91 and 0.92, precision 0.82), the decision-boundary effect described in Section 5.4. Bucketing changes which reviews are batched together and in what order, and length-homogeneous batches give less varied gradient estimates, which may contribute; these runs were not repeated over seeds, so the cause cannot be separated from run-to-run variation. AMP's training was almost identical to the fp32 run's (training loss 0.3910 against 0.3909 in epoch 1), so fp16 arithmetic cost no accuracy. Dynamic padding on its own saved no memory: its 271 MB equals that of the other fixed-length runs (271–272 MB), because peak memory is set by the longest batches, which still contain 256-token reviews. The baseline's 289 MB, the first training run in the session, most likely includes a one-off allocation. AMP lowers memory because the activations kept for the backward pass are stored in half precision.

A hardware-specific challenge: shape recompilation on Apple MPS. The notebook was first developed on an Apple M4 GPU (MPS backend), where the first dynamic-padding run was barely faster than fixed padding. MPS compiles a kernel graph for every new tensor shape, and exact per-batch lengths produced 88 distinct shapes; rounding batch lengths up to a multiple of 16 reduced this to 13. Table 9 compares the micro-benchmark on both devices; the M4 figures come from the earlier development run and are not part of the submitted notebook.

			T4 (CUDA), ms		M4 (MPS), ms	
Batch lengths	Shapes	Mean length	First pass	Steady	First pass	Steady
Fixed (256)	1	256.0	20.2	20.1	45.1	45.0
Rounded to a multiple of 16	13	208.3	15.6	15.5	39.0	35.9
Exact batch maxima	88	204.0	15.2	15.2	66.7	36.6

Table 9: Time per training step by padding strategy (micro-benchmark). Shapes: number of distinct batch lengths.

On the T4 there is no per-shape compilation cost: exact batch lengths are the fastest setting (15.2 ms per step, 25% faster than fixed padding), and rounding costs only 0.3–0.4 ms (2–3%, measured back to back in the same session) because its batches are slightly longer. On the M4, exact lengths were fast once compiled (36.6 ms) but 48% slower than fixed padding on the first pass (66.7 ms against 45.1 ms), because 88 shapes had to be compiled; rounding kept most of the saving with only 13 shapes. Rounding therefore solves an MPS-specific problem at almost no cost on CUDA.

6.3 Information lost by truncation (performance)

Challenge. 36.2% of training reviews exceed 256 tokens, and the baseline is less accurate on truncated test reviews (0.8265 versus 0.8884; Section 5.3). Many reviewers state their final verdict at the end, which truncation removes.

Solution and evidence. Two solutions were tested. Keeping the first 64 and the last 192 tokens of long reviews (head + tail) costs nothing extra (12.6 s per epoch against the baseline's

12.5 s, within timing noise) and raised test accuracy from 0.8669 to 0.8805; on truncated reviews, accuracy rose from 0.8265 to 0.8716. The efficiency gains of Section 6.2 made a longer context affordable: with max_len 512, dynamic padding and AMP, an epoch takes 11.6 s, less than the fp32 baseline's 12.5 s although it computes 1.30 times the attention work, but training memory rises to 617 MB. Test accuracy rises to 0.8706 (0.8544 on reviews longer than 256 tokens). Because single runs can mislead, both changes were repeated with seeds 1 and 2 (Table 10).

Configuration	Best val loss	Test accuracy	Test F1	Δ test acc.	Welch's t (p)
Baseline	0.2939 ± 0.0124	0.8599 ± 0.0117	0.8547 ± 0.0205	–	–
Head + tail	0.2670 ± 0.0014	0.8776 ± 0.0038	0.8756 ± 0.0088	+0.0177	2.50 (0.109)
max_len 512	0.2682 ± 0.0085	0.8744 ± 0.0035	0.8753 ± 0.0013	+0.0145	2.07 (0.155)

Table 10: Seed repeats (seeds 42, 1, 2): mean $\pm$ standard deviation; the t-test compares test accuracy with the baseline's.

Both gains are 1.5 to 1.8 percentage points, and every seed of head + tail and of max_len 512 has a higher test accuracy than every baseline seed. Welch's t-test is nevertheless not significant ($p = 0.109$ and 0.155), mainly because one baseline seed scored only 0.8464, which raises the baseline's standard deviation to 0.0117. A rank-based view is more favourable: if a change had no effect, the probability that all three of its runs beat all three baseline runs would be $1/\binom{6}{3} = 0.05$. The improvements are therefore consistent but not statistically established with three seeds. Head + tail is the better of the two fixes here: its mean test accuracy is slightly higher (0.8776 against 0.8744), it is more accurate on long reviews (0.8716 against 0.8544 for seed 42; Figure 4), and it needs less than half the training memory (271 MB against 617 MB). max_len 512 trains 8% faster per epoch only because it also uses dynamic padding and AMP. In these runs, keeping the end of a long review was at least as effective as reading more of it.

6.4 Overfitting

Challenge. The baseline overfits after the first epoch (Section 5.1): validation loss rises from 0.2902 to 1.3472 while training accuracy exceeds 0.99.

Solution and evidence. Raising dropout to 0.3 and adding decoupled weight decay of 0.1 (AdamW) lowered the epoch-10 validation loss to 0.8059 (Figure 7); with a linear warm-up and cosine decay added it was 0.9724. Epoch-10 validation accuracy rose from the baseline's 0.8398 to 0.8518 and 0.8608. The best checkpoints, however, were no better than the baseline's: test accuracy 0.8639 for the regularised run and 0.8326 for the run with the schedule.

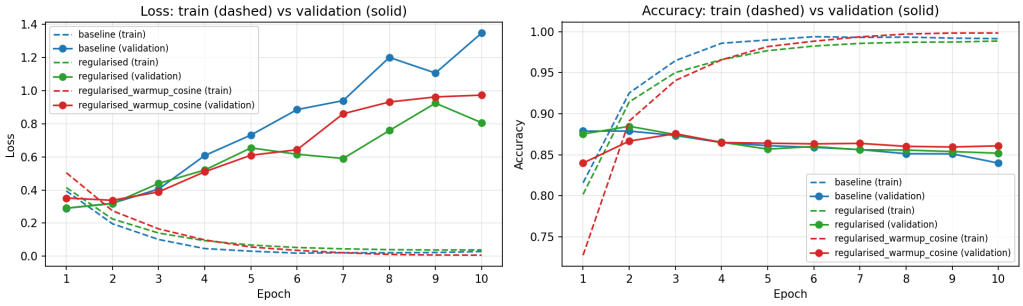


Figure 7: Training (dashed) and validation (solid) loss and accuracy per epoch, with and without regularisation.

Regularisation improves the later epochs but not the best checkpoint. With checkpoint selection, the model is taken at epoch 1 or 2, before overfitting sets in, so slowing the overfitting changes little: the regularised run's best validation loss (0.2904) equals the baseline's (0.2902), and its

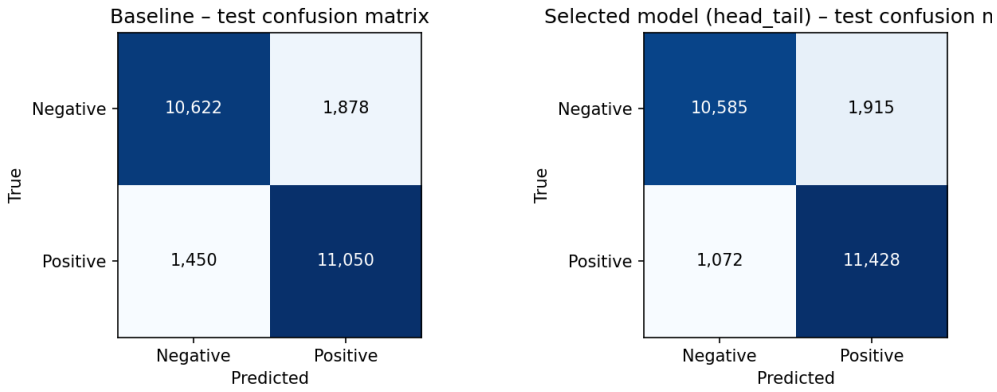
test accuracy (0.8639) is within the baseline's seed variation. The run with the schedule selected a clearly worse checkpoint (validation loss 0.3379, test accuracy 0.8326) that strongly favours negative predictions (precision 0.92, recall 0.73). Its warm-up ends after the first epoch, so the selected epoch-2 checkpoint was trained close to the peak learning rate. In the earlier MPS development run the same configuration reached 0.8630, so this single result is unstable rather than evidence that the schedule harms accuracy. The training curves show why regularisation cannot do more here: even with dropout 0.3 and weight decay, training accuracy passes 0.98 by epoch 6, so a model with 2.8 million parameters still memorises the 20,000 training reviews. Regularisation therefore matters mainly when training must run for a fixed number of epochs without checkpoint selection; preventing the overfitting itself would require stronger measures, such as pretrained embeddings or more training data.

6.5 Selected model

Selecting by the lowest best-checkpoint validation loss across all experiments chose head + tail truncation (epoch 1, validation loss 0.2665, validation accuracy 0.8888).

Head + tail, test set	Precision	Recall	F1	Support
Negative (0)	0.9080	0.8468	0.8764	12,500
Positive (1)	0.8565	0.9142	0.8844	12,500
Macro average	0.8823	0.8805	0.8804	25,000
Accuracy	0.8805 (TN 10,585, FP 1,915, FN 1,072, TP 11,428)			

Table 11: Class-wise test metrics of the selected model.



(a) Baseline (b) Selected model (head + tail)

Figure 8: Test confusion matrices.

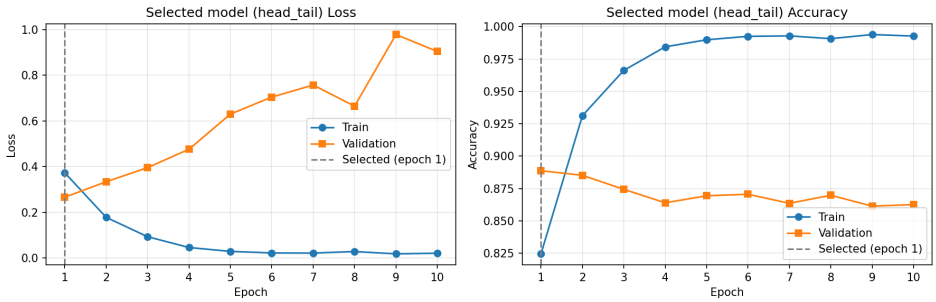


Figure 9: Training curves of the selected model.

Two results are reported. The baseline is the result for the configuration the assignment specifies, and it is used throughout Section 5. The final model is head + tail, because it has the lowest validation loss, the same criterion that selects the checkpoint within each run. The choice between the two truncation fixes is not clear-cut. On seed 42, head + tail leads max_len 512 by 0.009 in validation loss, but averaged over three seeds the lead is only 0.001 (0.2670 against 0.2682), far smaller than max_len 512's seed standard deviation (0.0085); in the earlier MPS development run, the same rule chose max_len 512. Head + tail is nevertheless the better choice on cost, since it trains at the baseline's cost and needs less than half of max_len 512's memory. Because the validation set was used to choose among eight configurations, the selected model's validation scores are optimistic (Section 5.2); the test set played no part in the choice.

6.6 Reproducibility across hardware

Comparing this run with the earlier development run on the Apple M4 shows which findings are robust. Three conclusions held on both devices, but only the first rests on several seeds: both truncation fixes beat the baseline by about 1.5 percentage points (three seeds each); regularisation did not improve the best checkpoint, and the padding mask had no measurable effect on accuracy (one run per device). Single-run results did not: dynamic padding's test accuracy (0.8469 on the M4, 0.8570 on the T4) and the direction of its errors, the result of the regularised run with a schedule (0.8630 against 0.8326) and the choice between the two truncation fixes all changed. Single-seed differences of one or two percentage points should therefore not be interpreted as effects.

7 Answers to the questions

Q1. How does self-attention work, and why is it important for sequence-based tasks?

Self-attention lets every token build its representation from a weighted mix of all tokens in the sequence, with the weights computed from the tokens themselves.

Each input vector $\mathbf{x}_i$ is projected by three learned matrices into a query $\mathbf{q}_i = W_q\mathbf{x}_i$, a key $\mathbf{k}_i = W_k\mathbf{x}_i$ and a value $\mathbf{v}_i = W_v\mathbf{x}_i$. The relevance of token j to token i is the dot product of $\mathbf{q}_i$ and $\mathbf{k}_j$, divided by $\sqrt{d_k}$. A softmax over j turns these scores into non-negative weights that sum to one, and the output for token i is the weighted sum of the values. In matrix form [1]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

The $\sqrt{d_k}$ scaling keeps the dot products from growing with the vector dimension, which would otherwise push the softmax into saturation and shrink its gradients. Padding positions are masked to a very large negative score before the softmax, so they receive zero weight. Multi-head attention runs several of these attention functions in parallel on lower-dimensional projections (here 4 heads of 32 dimensions). Each head can therefore capture a different relation, such as a negation and the word it modifies, and the heads are concatenated and combined by W_o.

Self-attention matters for sequence tasks for four reasons:

- **Long-range dependencies.** Any two tokens are connected in a single step, so a dependency such as "not ... at all good" does not have to pass through many recurrent steps. This avoids the vanishing-gradient problem that limits RNNs.
- **Context-dependent meaning.** Because the weights are computed from the input, the context each word draws on adapts to the sentence. The model can represent "good" differently in "not good" and in "very good".

- **Parallel computation.** All positions are processed at once with matrix multiplications, so training is much faster on GPUs than sequential recurrence.
- **Inspectable weights.** The attention weights can be inspected to see which tokens each representation drew on. In this small model, however, they are diffuse and favour function words (Section 5.3), so they are only a partial explanation of a prediction.

Self-attention is permutation-invariant: shuffling the tokens would give the same set of outputs. Positional encodings must therefore be added to supply word-order information.

Q2. If the model achieves poor accuracy, what are two potential reasons, and how could they be addressed?

Reason 1: information lost in the input representation. Truncation at 256 tokens removes the end of 36.2% of reviews, where the overall verdict is often stated. Rare words, misspellings and names outside the 20,000-word vocabulary all collapse into the single `<unk>` token, so their sentiment signal is lost.

Solutions. Increase the maximum length; dynamic padding and mixed precision make this affordable (with both, a 512-token model trained faster per epoch than the fp32 baseline), and in Section 6.3 it raised test accuracy from 0.8669 to 0.8706. Keep both the beginning and the end of long reviews (head-and-tail truncation), which in Section 6.3 raised it to 0.8805 at no extra cost. Use subword tokenisation such as BPE or WordPiece, so that unseen words are composed from known pieces instead of becoming `<unk>`.

Reason 2: overfitting and unstable optimisation of a model trained from scratch. The model has 2,825,218 parameters but only 20,000 training reviews, and its embeddings start from random values. It therefore memorises the training set: validation loss starts rising after epoch 1 while training loss keeps falling (Section 5.1). In addition, a fixed learning rate of 0.001 with no warm-up can make the early updates of a Post-LN Transformer noisy or oversized.

Solutions. Select the checkpoint with the lowest validation loss, as done here, or stop early. Strengthen regularisation with higher dropout or weight decay (AdamW). Add learning-rate warm-up followed by decay, and clip gradients. Initialise the embedding layer with pretrained word vectors such as GloVe, so that the model does not have to learn word meanings from 20,000 reviews alone.

The regularisation and schedule changes were tested in Section 6.4. They reduced late-epoch overfitting (epoch-10 validation loss 0.81 and 0.97, against 1.35 for the baseline) but did not improve the best checkpoint, so for this dataset checkpoint selection matters most, and the next step would be pretrained embeddings.

A further cause: implementation errors. A wrongly applied mask, a shape error when splitting the heads or a mismatched loss function can also cause poor accuracy without raising an error. The notebook guards against these with unit checks of the output shapes, of attention weights that sum to one, and of padding invariance.

8 Conclusion

The from-scratch Transformer reached 0.8669 test accuracy and 0.8691 F1 under the required configuration, and 0.8805 accuracy and 0.8844 F1 with head + tail truncation, the model selected by validation loss.

The largest accuracy gains came from addressing truncation. Head + tail and a 512-token context beat the baseline in every one of three seeds, by 1.8 and 1.5 percentage points on average, although with three seeds the differences are not statistically significant; head + tail achieved this at the baseline's cost. Dynamic padding with length bucketing made training 1.27× faster and cut the attention work by a third, and mixed precision raised the speed-up to 1.55× and cut training

memory by 23%; together they made a 512-token model cheaper to train than the fp32 baseline. Rounding batch lengths solved an MPS-specific compilation cost at almost no cost on CUDA. The padding mask had no measurable effect on accuracy and cost 10% of training time, but it is required for predictions that do not depend on padding, and regularisation improved the late epochs without improving the best checkpoint. Validation accuracy exceeded test accuracy in every run, and single-seed results changed between hardware, so only differences confirmed over several seeds should be relied on. The main remaining limitation is overfitting after the first epoch, which points to pretrained embeddings, subword tokenisation and an ensemble of the seed models as the most promising next steps.

AI use statement

Claude (Anthropic) was used for brainstorming. I used the tool to improve the clarity and structure of the report, understand some of the mathematical concepts, technical concepts.

- **Understanding library functions:** I used Claude to explain how several library functions work, including `register_buffer`, `torch.matmul`, `masked_fill`, `torch.finfo`, `nn.utils.clip_grad_norm_`, `LambdaLR`, `scipy.stats.ttest_ind` and `subprocess.run`. In addition to those, Claude was used to generate LaTeXsyntax for mathematical equations.

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems* 30.

[2] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y. and Potts, C. (2011). Learning word vectors for sentiment analysis. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics*, 142–150.

[3] Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.